

# A Two-Stage Deep Learning Pipeline for Automated Detection of Infectious Bovine Keratoconjunctivitis in Cattle

Iman Khazrak

Department of Computer Science

University Name

Email: imankhazrak@university.edu



**Abstract**—Infectious bovine keratoconjunctivitis (IBK), commonly known as pinkeye, is a highly contagious ocular disease in cattle that causes significant welfare and economic losses. Early automated detection from farm images can enable timely veterinary intervention. We present a two-stage deep learning pipeline: *Stage 1* localises the eye region using a YOLOv8 detector, and *Stage 2* classifies the cropped eye as healthy or diseased using four architectures—ResNet18, EfficientNet-B0, a baseline CNN+Transformer hybrid, and an enhanced CNN+Transformer variant with a class token and deeper head. Evaluated on 394 labelled eye crops (309 healthy, 85 pinkeye) under stratified 5-fold cross-validation, the Strong CNN+Transformer achieves the highest mean accuracy (95.94 %) and the highest mean F1-score on the minority pinkeye class (90.51 %); ResNet18 attains the highest mean ROC-AUC (98.67 %) among the four models in the primary benchmark. Augmentation ablations show architecture-dependent gains and trade-offs. These results support eye-localised classification as a feasible screen for this imbalanced, fine-grained imaging setting.

**Index Terms**—Pinkeye detection, YOLOv8, CNN+Transformer, class imbalance, data augmentation, cattle health monitoring, transfer learning.

## 1 METHODOLOGY

### 1.1 Overview of the Pipeline

The proposed system is a two-stage cascaded pipeline, illustrated in Fig. 1. In the first stage a YOLOv8-based object detector localises the eye region in each cattle head image. The detected bounding box is padded and cropped, producing a standardised eye-region image that forms the input to Stage 2. In the second stage, four deep classification models are trained and evaluated on the cropped eye images to discriminate between healthy eyes and eyes affected by IBK.

### 1.2 Dataset

#### 1.2.1 Eye Detection Dataset (Stage 1)

The YOLOv8 detector was trained on 117 cattle head images sourced from the Roboflow Eye Detection dataset (v3) [1]. Each image carries a single bounding-box annotation for the eye class in YOLO normalised format. All 117 annotation files were validated (pairing, coordinate ranges, class

ID = 0) prior to training. The dataset was split deterministically with seed 42 into training (80 %, 93 images), validation (10 %, 11 images), and test (10 %, 13 images), using the same stratification rule as in the logged detector training run.

#### 1.2.2 Classification Dataset (Stage 2)

The classification pool comprised 408 cattle head images with folder labels (healthy / pinkeye). After YOLO inference and cropping at confidence 0.25, 394 images yielded usable eye crops; 14 images had no passing detection and were set aside for manual review [2]. The Stage 2 dataset therefore contains 309 healthy and 85 pinkeye crops (78.4 % / 21.6 %). Images are resized to  $224 \times 224$  for classification.

### 1.3 Stage 1: Eye Localisation with YOLOv8

#### 1.3.1 Annotation Validation

The Roboflow export was checked with a validation script (format, bounds, class ID). All 117 labels passed.

#### 1.3.2 Detector Training

A YOLOv8n model pretrained on MS-COCO was fine-tuned on the 93-image training split using the Ultralytics YOLOv8 API [8]. Hyperparameters match the logged training run in the project detector report (Table 1).

TABLE 1  
YOLOv8n Training Hyperparameters (Stage 1)

| Parameter         | Value                                 |
|-------------------|---------------------------------------|
| Base model        | YOLOv8n (pretrained, MS-COCO)         |
| Epochs            | 100                                   |
| Image size        | $640 \times 640$                      |
| Batch size        | 16                                    |
| Optimizer         | AdamW (framework default LR schedule) |
| Training images   | 93                                    |
| Validation images | 11                                    |
| Test images       | 13                                    |
| Random seed       | 42                                    |

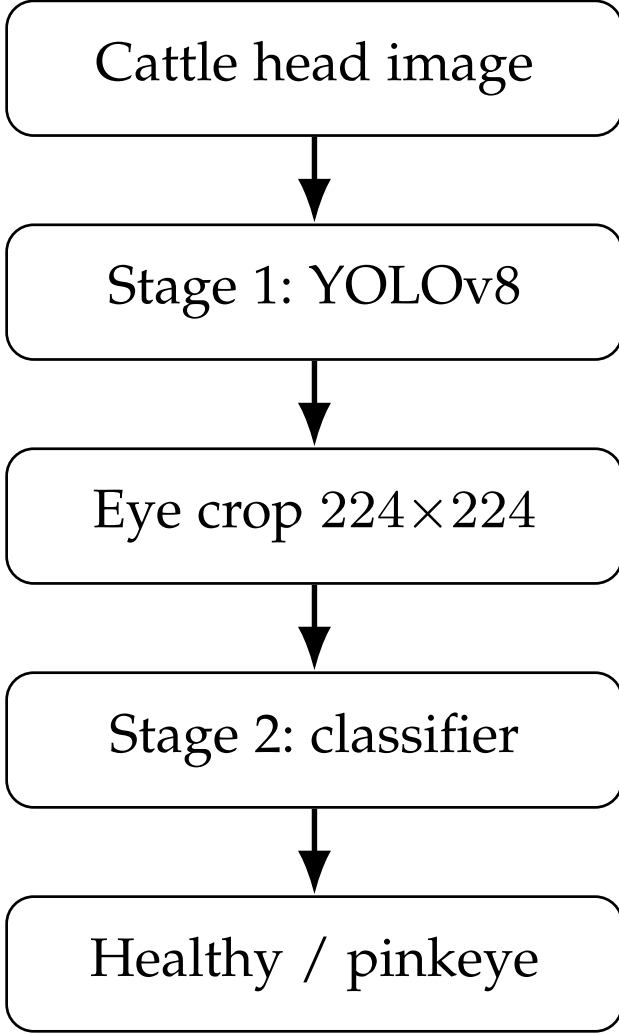


Fig. 1. High-level two-stage pipeline: localise the eye, then classify disease from the cropped region.

### 1.3.3 Eye Cropping

The best detector checkpoint was applied to all 408 classification images with confidence threshold 0.25 and symmetric padding 10% around each box (clamped to image bounds), matching the detector inference settings above (confidence 0.25, 10% symmetric padding). Figure 2 shows a representative head image from validation-style inference: the fine-tuned detector places a box on the eye with a class score, and the crop after padding is the resolution-matched input to Stage 2.

## 1.4 Stage 2: Classification Models

### 1.4.1 ResNet18 Baseline

ResNet18 [3] with ImageNet-1k weights was adapted by replacing the final fully connected layer with a  $512 \rightarrow 2$  head and fine-tuning the full network.

### 1.4.2 EfficientNet-B0 Baseline

EfficientNet-B0 [4] with ImageNet-1k weights was adapted with a  $1280 \rightarrow 2$  classifier head and end-to-end fine-tuning.

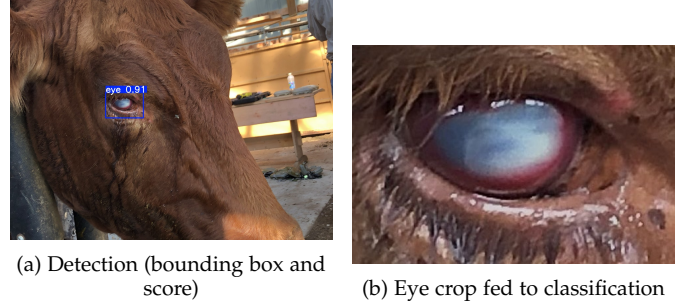


Fig. 2. Qualitative example of Stage 1 localisation after fine-tuning: (a) predicted eye region on a cattle head image; (b) cropped eye with padding, resized to  $224 \times 224$  before Stage 2.

### 1.4.3 Baseline CNN+Transformer Hybrid

The baseline hybrid feeds final-stage CNN feature maps through a  $1 \times 1$  projection, flattens spatial locations into tokens, applies a Transformer encoder, mean-pools tokens, and classifies with a linear head in our implementation.

### 1.4.4 Strong CNN+Transformer

The Strong variant uses an EfficientNet-B0 backbone (features-only),  $1 \times 1$  projection with BatchNorm and GELU, patch tokens with a learnable CLS token and learnable positional embeddings, a pre-norm Transformer encoder ( $L=3$ ,  $h=4$ ,  $d=256$ ), and an MLP classification head on the CLS output [6]. Hyperparameters are summarised in Table 2.

TABLE 2  
Strong CNN+Transformer Architectural Hyperparameters

| Parameter               | Value                        |
|-------------------------|------------------------------|
| CNN backbone            | EfficientNet-B0 (pretrained) |
| Embedding dimension $d$ | 256                          |
| Transformer heads $h$   | 4                            |
| Transformer layers $L$  | 3                            |
| MLP ratio               | 4.0                          |
| Dropout                 | 0.1                          |
| Attention dropout       | 0.1                          |
| Token norm              | Pre-LayerNorm                |
| Activation              | GELU                         |

## 1.5 Training Protocol

### 1.5.1 Stratified 5-Fold Cross-Validation

With 394 crops, we used stratified 5-fold cross-validation (seed 42), preserving the healthy/pink-eye ratio in each fold. Each fold uses four parts for training and one for validation; metrics are summarised as mean  $\pm$  standard deviation across folds.

### 1.5.2 Loss Function and Class Imbalance

Training used weighted cross-entropy with class weights  $w_c = N/(K n_c)$  over the *training-fold* sample counts (inverse frequency within each fold). This upweights the minority pinkeye class relative to healthy samples.

### 1.5.3 Optimiser and Early Stopping

Optimisation used AdamW [5] with learning rate  $10^{-4}$  and weight decay  $10^{-4}$ . Early stopping monitored validation F1 on the pinkeye class with patience 8 and minimum improvement  $\delta = 10^{-4}$ .

### 1.5.4 Cluster Benchmark and Ablation Protocol

All Step 2 benchmarks and augmentation studies were run as batch GPU jobs on a shared cluster managed by the SLURM workload scheduler. Every study used the same stratified 5-fold split (seed 42), weighted cross-entropy, AdamW learning rate  $10^{-4}$  (weight decay  $10^{-4}$ ), batch size 8, early stopping on validation  $F1_{PE}$  with patience 8 (minimum improvement  $\delta=10^{-4}$ ), and CUDA acceleration with the centre’s module stack. Training was capped at five epochs per fold unless stopped earlier by early stopping. Default settings used for ad hoc interactive runs may differ; all figures and tables in Section 2 follow this cluster protocol.

### 1.5.5 Augmentation Policies (Reported Experiments)

Four training augmentations were compared. (i) **Full augmentation** defines the primary benchmark: random resized crop, horizontal flip, rotation ( $\pm 10^\circ$ ), colour jitter, and mild affine transforms, with validation and test limited to resize and ImageNet normalisation [6]. (ii) **No-colour augmentation** retained flip, rotation, and light Gaussian noise but omitted colour jitter. (iii) **Flip and rotation only** removed Gaussian noise relative to (ii). (iv) **Minority-only flip and rotation** applied geometric augmentation only to pinkeye-labelled training crops.

**1.5.5.1 Reproducibility note:** The publicly released training code implements resize, horizontal flip, and rotation for training—a subset of (i)–(ii). The richer policies above correspond to the frozen experiment configuration used to produce Section 2; reproducing those runs exactly may require the same transform definitions and software versions as in the original study.

## 1.6 Evaluation Metrics

For each validation fold we report accuracy, balanced accuracy, precision/recall/F1 for the pinkeye (minority) class, and ROC-AUC (one-vs-rest using predicted pinkeye probability). The primary score for comparing augmentation settings is  $F1_{PE}$  (F1 on pinkeye).

## 1.7 Ablation Experiment Design

We compared the four augmentation policies in Section 1.5.5 under the cluster benchmark protocol in Section 1.5.4, keeping the same folds, loss, and stopping rule within each compared study. Model sets varied slightly by experiment (all four models for configurations (i)–(ii); subsets for (iii)–(iv) as listed in the tables).

## 2 RESULTS

### 2.1 Dataset Distribution

Figure 3 summarises the Stage 2 class counts used in all experiments.

### 2.2 Main Classification Benchmark

Table 3 lists 5-fold mean  $\pm$  std performance under the full augmentation configuration (policy (i)). Figure 4 visualises mean  $F1_{PE}$  with error bars using the same fold-wise summaries as Table 3.

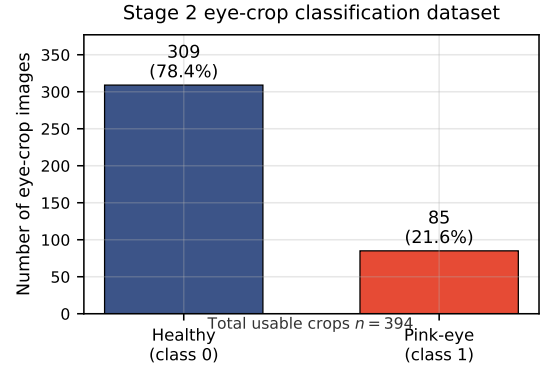


Fig. 3. Class distribution of usable Stage 2 eye crops ( $n=394$ ): 309 healthy (78.4%) and 85 pinkeye (21.6%), matching folder labels after YOLO cropping.

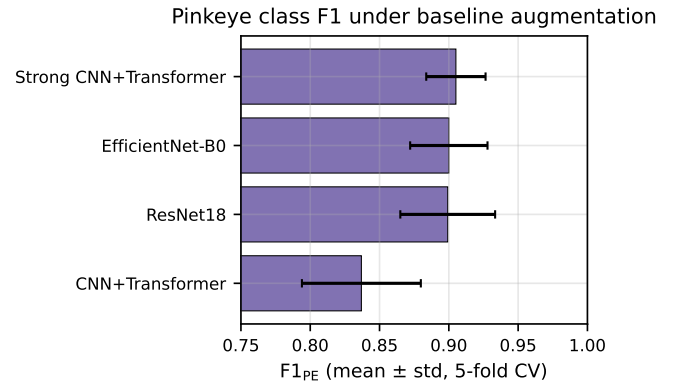


Fig. 4. Mean  $\pm$  std  $F1_{PE}$  under full augmentation, replotted from the same statistics as Table 3.

## 2.3 Fold-Level Behaviour and Confusion Matrix

Table 4 gives per-fold accuracy and  $F1_{PE}$  for ResNet18 and Strong CNN+Transformer. Table 5 aggregates confusion counts across folds for Strong CNN+Transformer.

Fold-level confusion matrices, ROC curves, and learning curves were saved with the original GPU experiments but are omitted here for brevity.

## 2.4 Augmentation Comparisons

### 2.4.1 Removing Colour Augmentation

Table 6 contrasts policy (i) (full augmentation) with policy (ii) (no colour) using five-fold mean  $F1_{PE}$  and accuracy.

Removing colour augmentation improved ResNet18 but reduced  $F1_{PE}$  for EfficientNet-B0 and both hybrid models, indicating an interaction between architecture and augmentation strength.

### 2.4.2 Gaussian Noise Ablation

Table 7 compares no-colour training with and without Gaussian noise by juxtaposing policy (ii) and policy (iii) means under the same protocol.

TABLE 3  
Stage 2 classification under full augmentation (5-fold mean  $\pm$  std). Best per column in bold.

| Model                         | Accuracy                              | Bal. Acc.                             | Precision <sub>PE</sub>               | Recall <sub>PE</sub>                  | F1 <sub>PE</sub>                      | ROC-AUC                               |
|-------------------------------|---------------------------------------|---------------------------------------|---------------------------------------|---------------------------------------|---------------------------------------|---------------------------------------|
| ResNet18                      | 0.9569 $\pm$ 0.0144                   | 0.9342 $\pm$ 0.0289                   | 0.9094 $\pm$ 0.0619                   | 0.8941 $\pm$ 0.0644                   | 0.8992 $\pm$ 0.0342                   | <b>0.9867 <math>\pm</math> 0.0117</b> |
| EfficientNet-B0               | 0.9568 $\pm$ 0.0114                   | <b>0.9383 <math>\pm</math> 0.0297</b> | 0.8980 $\pm$ 0.0393                   | <b>0.9059 <math>\pm</math> 0.0671</b> | 0.9000 $\pm$ 0.0279                   | 0.9829 $\pm$ 0.0122                   |
| CNN+Transformer               | 0.9239 $\pm$ 0.0235                   | 0.9132 $\pm$ 0.0215                   | 0.7934 $\pm$ 0.0928                   | 0.8941 $\pm$ 0.0492                   | 0.8369 $\pm$ 0.0429                   | 0.9681 $\pm$ 0.0327                   |
| <b>Strong CNN+Transformer</b> | <b>0.9594 <math>\pm</math> 0.0106</b> | 0.9358 $\pm$ 0.0229                   | <b>0.9251 <math>\pm</math> 0.0795</b> | 0.8941 $\pm$ 0.0644                   | <b>0.9051 <math>\pm</math> 0.0214</b> | 0.9850 $\pm$ 0.0024                   |

PE = pinkeye (minority class). Metrics aggregated from five-fold validation.

TABLE 4  
Per-fold accuracy and F1<sub>PE</sub> (full augmentation).

| Fold | ResNet18 |                  | Strong CNN+T |                  |
|------|----------|------------------|--------------|------------------|
|      | Acc.     | F1 <sub>PE</sub> | Acc.         | F1 <sub>PE</sub> |
| 1    | 0.9494   | 0.8889           | 0.9494       | 0.8889           |
| 2    | 0.9620   | 0.9143           | 0.9747       | 0.9412           |
| 3    | 0.9747   | 0.9412           | 0.9494       | 0.8889           |
| 4    | 0.9367   | 0.8485           | 0.9620       | 0.9032           |
| 5    | 0.9615   | 0.9032           | 0.9615       | 0.9032           |
| Mean | 0.9569   | 0.8992           | 0.9594       | 0.9051           |
| Std  | 0.0144   | 0.0342           | 0.0106       | 0.0214           |

TABLE 5  
Aggregated confusion matrix for Strong CNN+Transformer (sum over validation folds; total samples = 394).

| True \ Pred | Healthy | Pink-eye |
|-------------|---------|----------|
| Healthy     | 302     | 7        |
| Pink-eye    | 9       | 76       |

### 2.4.3 Minority-Only Augmentation

Table 8 summarises mean F1<sub>PE</sub> under no-colour+noise, all-class flip+rotation, and minority-only flip+rotation under the same protocol.

### 2.4.4 Best Model per Configuration

Table 9 condenses the best mean accuracy and F1<sub>PE</sub> per augmentation regime.

Under the primary full-augmentation benchmark, Strong CNN+Transformer delivers the highest mean accuracy and F1<sub>PE</sub>. Under the no-colour policy, ResNet18 achieves a higher mean F1<sub>PE</sub> (0.9190) but with larger fold-wise dispersion ( $\pm 0.0374$  vs.  $\pm 0.0214$  for Strong CNN+Transformer under full augmentation).

## 2.5 DDPM-Based Synthetic Augmentation (Pilot)

To probe generative remedies for class imbalance, we trained a class-conditional denoising diffusion probabilistic model (DDPM) [9] to synthesise eye crops labelled *healthy* or *pinkeye*, matching the Stage 2 taxonomy. The diffusion training configuration used batch size 16,  $T=10,000$  noise steps, and 2048 epochs. Figure 5 shows representative real crops from the classification pool next to synthetic draws from the trained generators for each class.

The intended workflow is to screen synthetic samples for anatomical and pathology plausibility, then add accepted images to training folds in a controlled mix with real data, re-run the stratified 5-fold Stage 2 benchmarks, and compare

TABLE 6  
Effect of removing colour-related augmentation: mean F1<sub>PE</sub> and accuracy (5-fold mean).

| Model           | F1 <sub>PE</sub> (full) | F1 <sub>PE</sub> (no-col) | $\Delta$ F1    | $\Delta$ Acc |
|-----------------|-------------------------|---------------------------|----------------|--------------|
| ResNet18        | 0.8992                  | <b>0.9190</b>             | <b>+0.0198</b> | +0.0076      |
| EfficientNet-B0 | <b>0.9000</b>           | 0.8721                    | −0.0279        | −0.0152      |
| CNN+Transformer | 0.8369                  | 0.8272                    | −0.0097        | −0.0001      |
| Strong CNN+T    | <b>0.9051</b>           | 0.8672                    | −0.0379        | −0.0153      |

TABLE 7  
Gaussian noise ablation: mean F1<sub>PE</sub>.  $\Delta$  is flip+rotation (no noise) minus no-colour+noise.

| Model           | No-col+noise | Flip+rot      | $\Delta$ F1 <sub>PE</sub> |
|-----------------|--------------|---------------|---------------------------|
| ResNet18        | 0.9190       | 0.8979        | −0.0210                   |
| EfficientNet-B0 | 0.8721       | 0.8518        | −0.0202                   |
| Strong CNN+T    | 0.8672       | <b>0.8788</b> | <b>+0.0117</b>            |

minority-class recall and F1<sub>PE</sub> against real-only training. Those mixed training runs are preparatory: all tables above report models trained only on real eye crops.

## 3 DISCUSSION

### 3.1 Architecture Rankings and Augmentation Interactions

Strong CNN+Transformer leads on mean accuracy and F1<sub>PE</sub> when the richer augmentation policy is used, while ResNet18 achieves the best mean ROC-AUC in Table 3 and the best mean F1<sub>PE</sub> under the no-colour regime (Table 9). The ablations indicate that colour and affine regularisation in policy (i) particularly benefit the deeper CNN and hybrid models, whereas the shallow ResNet benefits when colour perturbations are omitted.

### 3.2 Qualitative Attribution

Gradient-based Grad-CAM visualisations [7] were computed for the CNN baselines during training. Illustrative overlays are not reproduced in this manuscript; qualitatively, saliency tended to align with conjunctival and peripheral corneal regions on pinkeye crops, consistent with clinical presentation.

### 3.3 Limitations and Future Work

The dataset size (85 pinkeye crops) limits generalisation claims. A class-conditional DDPM for synthetic augmentation was trained [9]; Figure 5 illustrates sample quality relative to real crops. Systematic Stage 2 fine-tuning with

TABLE 8  
Minority-only versus all-class geometric augmentation (mean  $F1_{PE}$ ).

| Model           | No-col+noise  | Flip+rot (all) | Flip+rot (min.) | Best setting   |
|-----------------|---------------|----------------|-----------------|----------------|
| ResNet18        | <b>0.9190</b> | 0.8979         | 0.9060          | no-col+noise   |
| EfficientNet-B0 | <b>0.8721</b> | 0.8518         | 0.8624          | no-col+noise   |
| Strong CNN+T    | 0.8672        | <b>0.8788</b>  | 0.8582          | flip+rot (all) |

curated synthetic augmentation and reporting against the benchmark protocol remains future work. Broader directions include larger multi-site collections, severity grading, and external validation.

## ACKNOWLEDGEMENTS

The authors acknowledge computing resources provided by [HPC Centre]. Step 1 and Step 2 GPU jobs used the institution’s SLURM-managed cluster with CUDA modules provisioned by the computing centre.

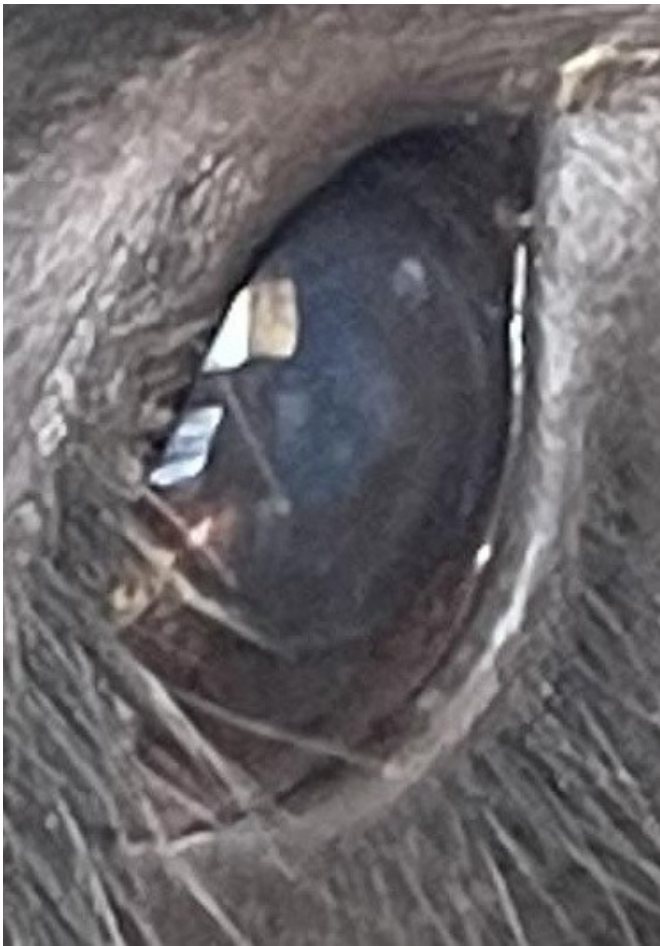
## REFERENCES

- [1] I. Khazrak, “Eye Detection Dataset (v3),” Roboflow Universe, 2026. [Online]. Available: <https://universe.roboflow.com/imans-workspace-rsc4n/eye-detection-fayd6>
- [2] Internal technical memorandum, Cattle Pinkeye Detection project, Mar. 2026 (dataset inventory and crop statistics).
- [3] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *Proc. IEEE/CVF CVPR*, 2016, pp. 770–778.
- [4] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *Proc. ICML*, 2019, pp. 6105–6114.
- [5] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” in *Proc. ICLR*, 2019.
- [6] R. Wightman, “PyTorch Image Models,” GitHub, 2019. [Online]. Available: <https://github.com/rwightman/pytorch-image-models>
- [7] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, “Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization,” in *Proc. IEEE ICCV*, 2017, pp. 618–626.
- [8] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLOv8,” GitHub, 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [9] J. Ho, A. Jain, and P. Abbeel, “Denoising Diffusion Probabilistic Models,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 6840–6851.

TABLE 9  
Best mean accuracy /  $F1_{PE}$  by augmentation regime (5-fold means).

| Configuration                       | Best model             | Accuracy            | $F1_{PE}$           |
|-------------------------------------|------------------------|---------------------|---------------------|
| Full augmentation (colour + affine) | Strong CNN+Transformer | $0.9594 \pm 0.0106$ | $0.9051 \pm 0.0214$ |
| No colour (+ Gaussian noise)        | ResNet18               | $0.9645 \pm 0.0165$ | $0.9190 \pm 0.0374$ |
| Geometric only (flip + rotation)    | ResNet18               | $0.9543 \pm 0.0147$ | $0.8979 \pm 0.0312$ |
| Minority-only augmentation          | ResNet18               | $0.9594 \pm 0.0105$ | $0.9060 \pm 0.0282$ |





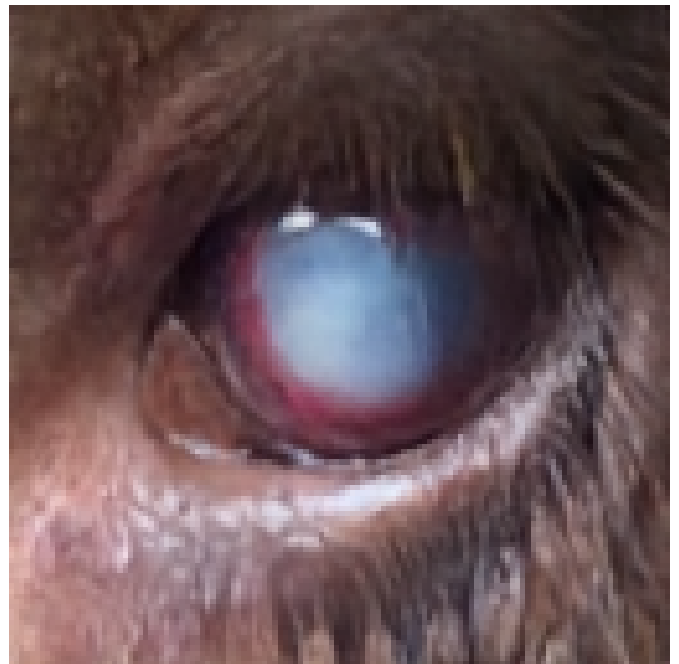
(a) Healthy crop (real)



(b) Healthy (DDPM synthetic)



(c) Pinkeye crop (real)



(d) Pinkeye (DDPM synthetic)

Fig. 5. Qualitative comparison of real Stage 2 crops and class-conditional DDPM samples (pilot augmentation study). Training used batch size 16,  $T=10,000$ , and 2048 epochs.